React Practice Lab

A focused set of hands-on challenges for building strong React fundamentals.

TOPICS	FORMAT	APPROACH
Lists & Keys · Composition useContext · useReducer	6 practical exercises with success criteria	Build · Observe Explain · Refactor

How to use this workbook

- Start with the smallest working solution; avoid unnecessary abstractions.
- Use meaningful component, prop, and action names.
- Test add, remove, update, and reorder behavior in the browser.
- Read console warnings and explain what caused them.
- Complete the core requirements before attempting the stretch challenge.

01 / Lists & Keys — Student Directory

Level: Beginner · Suggested time: 15 minutes

MentorBridge needs a directory of students enrolled in Cohort #3.

STARTER DATA

```
const students = [
{ id: "s1", name: "Arun", track: "React", score: 85 },
{ id: "s2", name: "Priya", track: "React", score: 92 },
{ id: "s3", name: "Kumar", track: "React", score: 76 },
];
```

- Render each student through a reusable StudentCard component.
- Use map() and each student’s stable id as the key.
- Display name, track, and score; add sorting by score, highest first.

Success criteria: Sorting works, every student renders, and there are no missing-key warnings.

Stretch challenge: Insert a student at the beginning and verify that existing rows retain their identity.

Practice challenges

02 / Component Composition — Cohort Dashboard

Build a dashboard from reusable parts

BEGINNER · 20 minutes

Goal Combine small components instead of putting all the markup in one component.

TASKS

- Create Header, CohortSummary, StudentList, StudentCard, and Footer.
- Pass student information through props.
- Create a reusable Card component that accepts a title and children.
- Use Card for both the cohort summary and learning-progress panel.

SUCCESS CRITERIA

- Each component has one clear responsibility.
- StudentCard is reused for every student.
- Card renders different content through children.

Stretch challenge Add compact and expanded layouts without duplicating student data.

03 / useContext — Shared Theme and Cohort

INTERMEDIATE · 20–25 minutes

Goal Share values with deeply nested components without passing props through every layer.

TASKS

- Create an AppContext using createContext().
- Store theme (light/dark) and cohortName in the provider value.
- Build a ThemeToggle component that changes the theme.
- Read theme and cohortName inside a nested StudentCard using useContext().

SUCCESS CRITERIA

- Toggling the theme updates relevant components.
- Nested components access cohortName without prop drilling.
- Provider values and update functions are used consistently.

Stretch challenge Separate unrelated contexts if changing one value causes unnecessary consumer updates.

04 / useReducer — Student Progress Tracker

INTERMEDIATE · 25–30 minutes

Goal Manage related state transitions through explicit actions and a reducer.

TASKS

- Represent students with id, name, score, and status.
- Implement SCORE_UPDATED, STATUS_TOGGLED, and STUDENT_ADDED actions.
- Use useReducer(reducer, initialStudents).
- Dispatch actions from buttons and a simple add-student form.
- Return new arrays and objects; never mutate existing state.

SUCCESS CRITERIA

- Each action updates only the intended record or collection.
- Unknown actions return the existing state.
- The UI updates after each dispatch.

Stretch challenge Add SCORE_RESET and reject scores outside 0–100.

Integration & debugging

05 / Combine useContext + useReducer

INTERMEDIATE · 30 minutes

Goal Share a single reducer-managed student state across multiple components.

TASKS

- Create StudentProvider to own the useReducer state.
- Expose students and dispatch through Context.
- Build StudentSummary, StudentList, and ScoreControls.
- Ensure the summary and list reflect each dispatched action.
- Avoid keeping a competing copy of student state in child components.

SUCCESS CRITERIA

- Components dispatch actions without prop drilling.
- There is one source of truth for student data.
- The reducer remains a pure function.

Stretch challenge Create a useStudents() hook that throws a clear error outside its provider.

06 / Debug the Keys

INTERMEDIATE · 15 minutes

Goal See why stable keys matter when list items are inserted, removed, or reordered.

TASKS

- Render a list using array indexes as keys.
- Give each row local state, such as an editable note or checkbox.
- Insert a student at the beginning and observe row state.
- Replace index keys with stable IDs and repeat the experiment.

SUCCESS CRITERIA

- Explain how keys help React match items between renders.
- Stable IDs preserve intended identity when items move.
- Avoid random values generated during render as keys.

Stretch challenge Add a remove-student action and test identity again.

Final review checklist

☐ I can explain why list items need stable, unique sibling keys.

☐ I can compose reusable components using props and children.

☐ I can explain when Context is useful and when prop drilling is acceptable.

☐ I can write a pure reducer and dispatch meaningful actions.

☐ I can combine Context and useReducer without duplicating state.

☐ I can test add, remove, update, and reorder flows.

THE MENTORBRIDGE METHOD
Build it. Observe it. Explain it. Refactor it.